



수백억의 손실을 막는 단 하나의 신호

AutoEncoder 기반 설비 진동신호 결함 진단 시스템

훈련생	역할	담당 업무	
박기랑	팀장	 데이터 정제 및 모델링	 자료제작 및 발표
유봉철	팀원	 웹 플랫폼 구현	 서버/클라우드 구현
박정환	팀원	 프론트엔드	 백엔드
임성택	팀원	 프론트엔드	 백엔드

구분	기간	활동		비고
사전 기획	12/15(월) ~ 12/20(금)	 프로젝트 기획/주제 선정	 기획안 작성	아이디어 선정
데이터 수집	12/22(월) ~ 12/23(화)	 필요 데이터 정의 및 수집	 외부 데이터 수집	데이터 수집
데이터 전처리	12/23(화) ~ 12/24(수)	 데이터 정제 및 정규화		
모델링	12/26(금) ~ 12/30(화)	 모형 구현		
서비스 구축	12/31(수) ~ 01/07(수)	 웹 서비스 시스템 설계	 웹 플랫폼 구현	웹서비스 구축, 오류 수정
총 개발기간	12/15(월) ~ 01/08(목)(총4주)			

미세한 이상이 치명적인 사고로 이어지는 산업 현장

항공기, 로켓, 산업용 로봇과 같은 초고가·초고위험 시스템에서는
볼트 하나, 베어링 하나의 미세한 이상도 대형 사고로 직결될 수 있습니다.



■ **막대한 금전적 손실:** 수백억~수천억 원의 장비 비용 및 발사/비행 중단에 따른 손실



■ **인명 피해 및 신뢰도 하락:** 돌이킬 수 없는 인명 피해와 기업의 기술 신뢰도 손실



■ **기회 비용:** 장비 전량 폐기 및 프로젝트 실패로 인한 기회 상실

왜 AI가 필요한가:

원본 진동 신호는 육안으로 구분이 어렵습니다

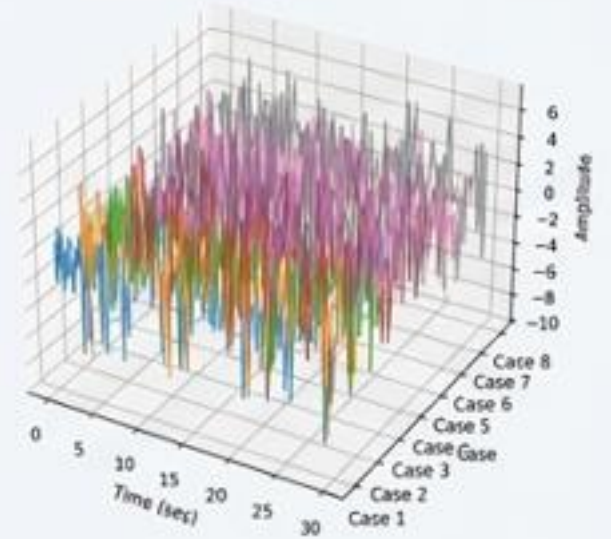


- 원본 진동 신호(시간 영역)는 노이즈처럼 보이며, 특정 규칙이나 임계값으로 '볼트 풀림' 상태를 정의하기 매우 어렵습니다.
- 16개 케이스별 신호 패턴이 미세하게 다르지만, 일관된 규칙을 사람이 찾아내기는 사실상 불가능합니다. 이는 AI 기반 패턴 인식이 필수적인 이유입니다.

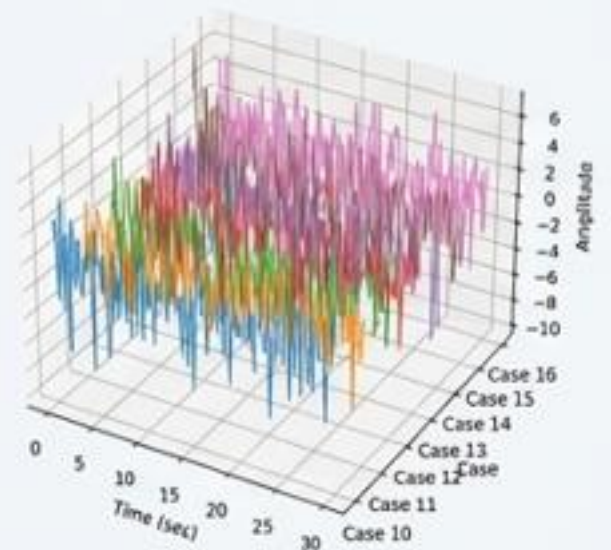
	A	B
1	1.602863	-13.2555
2	0.469041	-12.5714
3	0.410609	-11.2503
4	-1.06731	-11.9489
5	-1.14522	-11.7346
6	-1.11217	-9.92082
7	-1.00711	-11.2274
8	-0.80526	-14.3229
9	-1.14109	-12.9451
10	-0.66596	-10.8023
11	-0.90736	-12.3999
12	-1.6298	-12.7247
13	-1.8287	-10.6381
14	0.20151	10.8108



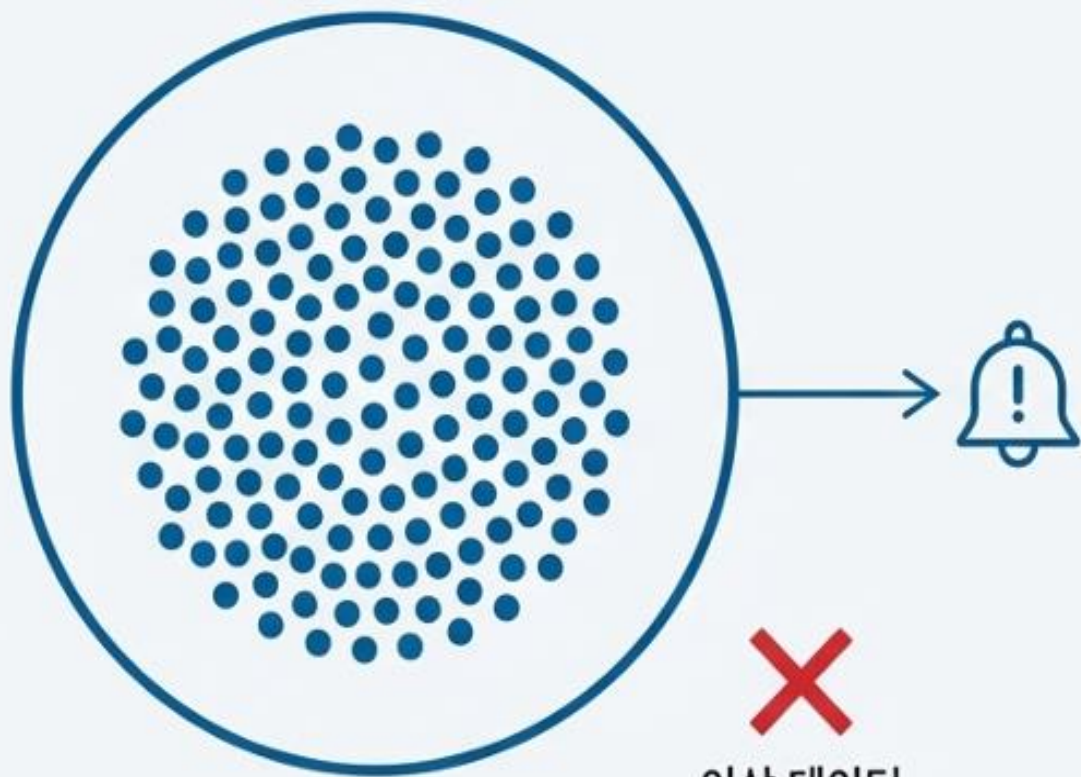
Raw Vibration Signal (3D) - Cases 1-8



Raw Vibration Signal (3D) - Cases 10-16



정상 데이터
(Case 1)

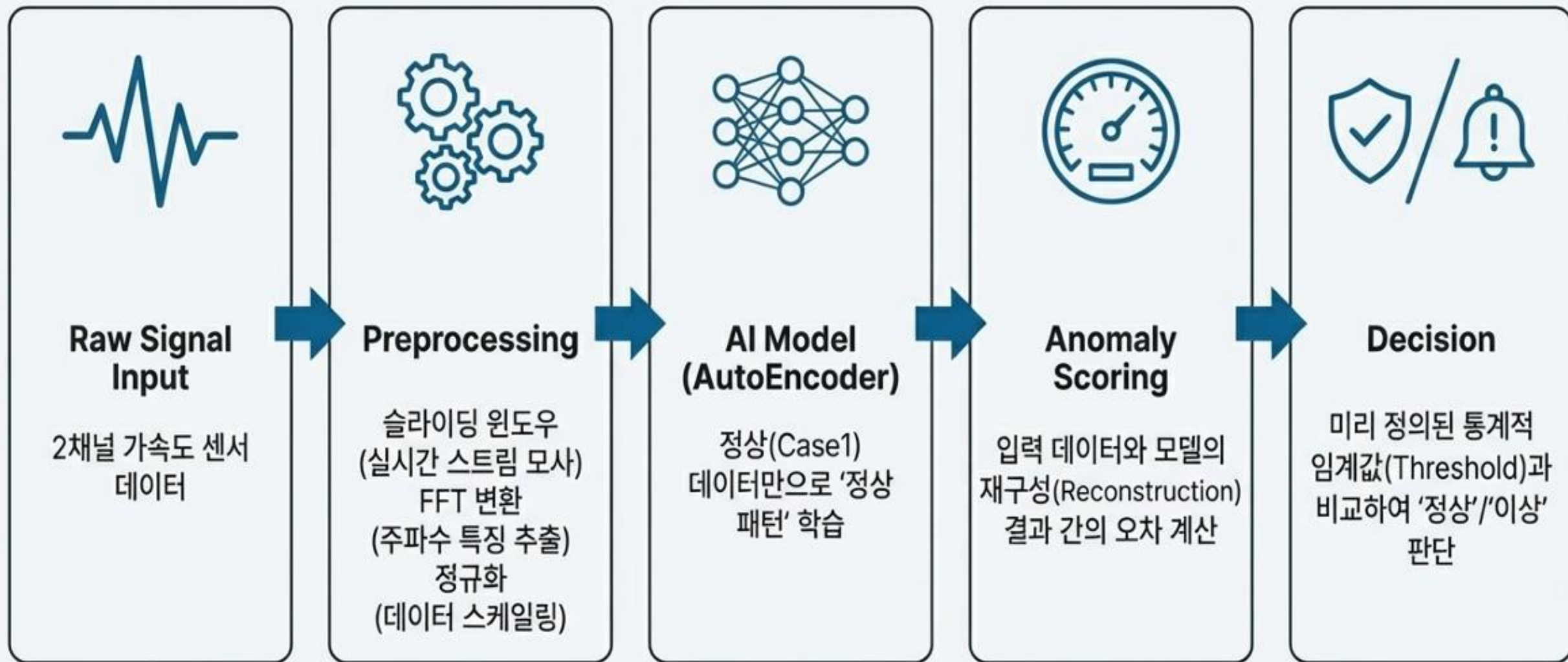


이상 데이터
(Unknown)

핵심 제약: '고장' 데이터 없이 '정상' 데이터만으로 이상을 탐지해야 합니다

- **현실적 문제:** 실제 산업 현장에서는 장비가 대부분 정상적으로 작동하므로, '고장' 또는 '이상' 상태의 데이터를 충분히 확보하기 어렵습니다.
- **해결 방향:** 따라서, 수집이 용이한 '정상' 상태의 데이터 패턴만을 학습하고, 이 패턴에서 벗어나는 신호를 '이상'으로 탐지하는 One-Class Learning 방식이 필요합니다.
- **선택 모델:** 본 프로젝트는 이러한 요구사항에 최적화된 AutoEncoder 기반의 이상탐지 모델을 채택했습니다. **`Case1`** (All-tight)을 정상 상태로 정의하고 학습을 진행합니다.

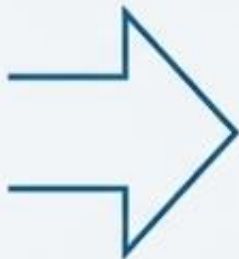
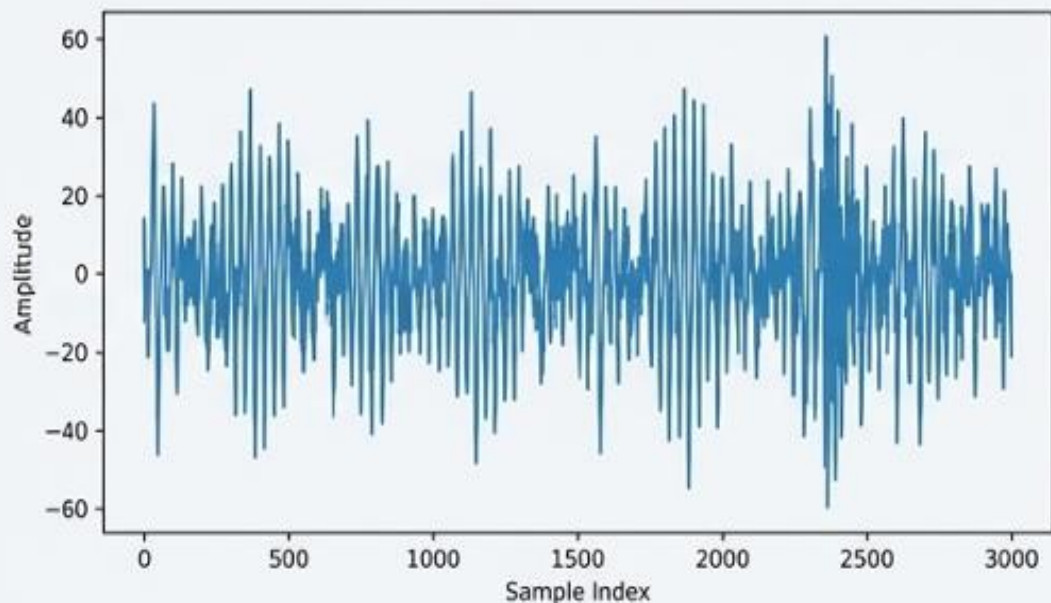
End-to-End 솔루션: 원본 신호에서 이상 판단까지의 전체 파이프라인



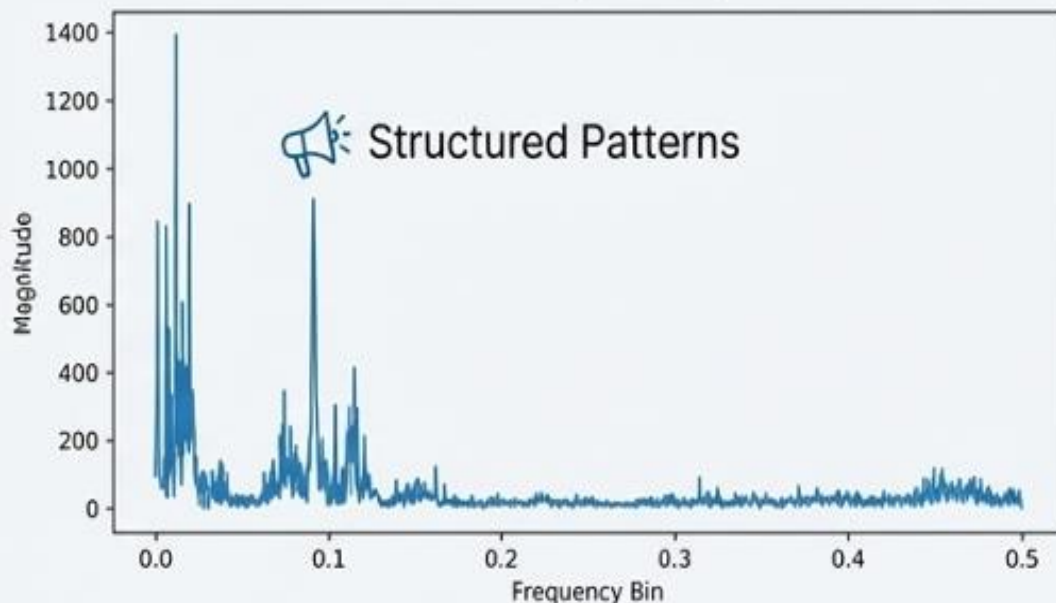
Step 1. 특징 추출: 시간이 아닌 주파수에서 구조적 패턴을 찾습니다

- 진동 신호의 구조적 결함(볼트 풀림 등)은 시간 영역보다 주파수 영역에서 더 명확한 차이를 보입니다.
- 실시간 적용을 가정하여, 신호를 짧은 구간(Window)으로 나눈 뒤 각 구간마다 FFT(고속 푸리에 변환)를 적용하여 주파수별 에너지 분포를 특징으로 사용합니다.
- 추출된 특징은 $\log(1 + \text{magnitude})$ 변환을 통해 안정화됩니다.

시간 영역 (Time Domain)



주파수 영역 (Frequency Domain)



문제는 ‘고장 데이터’가 존재하지 않는다는 것입니다

기존 점검 방식의 한계

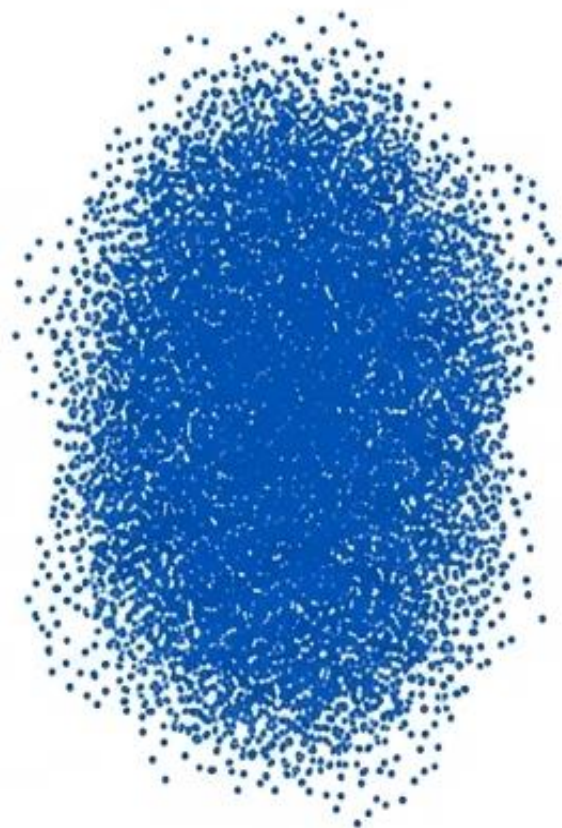
육안 점검, 주기적 분해 검사, 명확한 임계값 기반 센서 알람과 같은 방식은 ‘정상과 비정상의 경계에 있는 미세한 변화’는 놓치기 쉽습니다.

데이터의 근본적인 제약

정상 상태의 운영 데이터는 풍부하게 존재합니다. 하지만, 대형 사고로 이어지는 **실제 고장 데이터**는 거의 없거나 존재하지 않습니다.

따라서, 레이블된 고장 데이터를 필요로 하는 지도학습(Supervised Learning) 기반 접근은 원천적으로 불가능합니다.

정상 상태 운영 데이터
(Abundant Normal Data)



실제 고장 데이터
(Non-existent Failure Data)

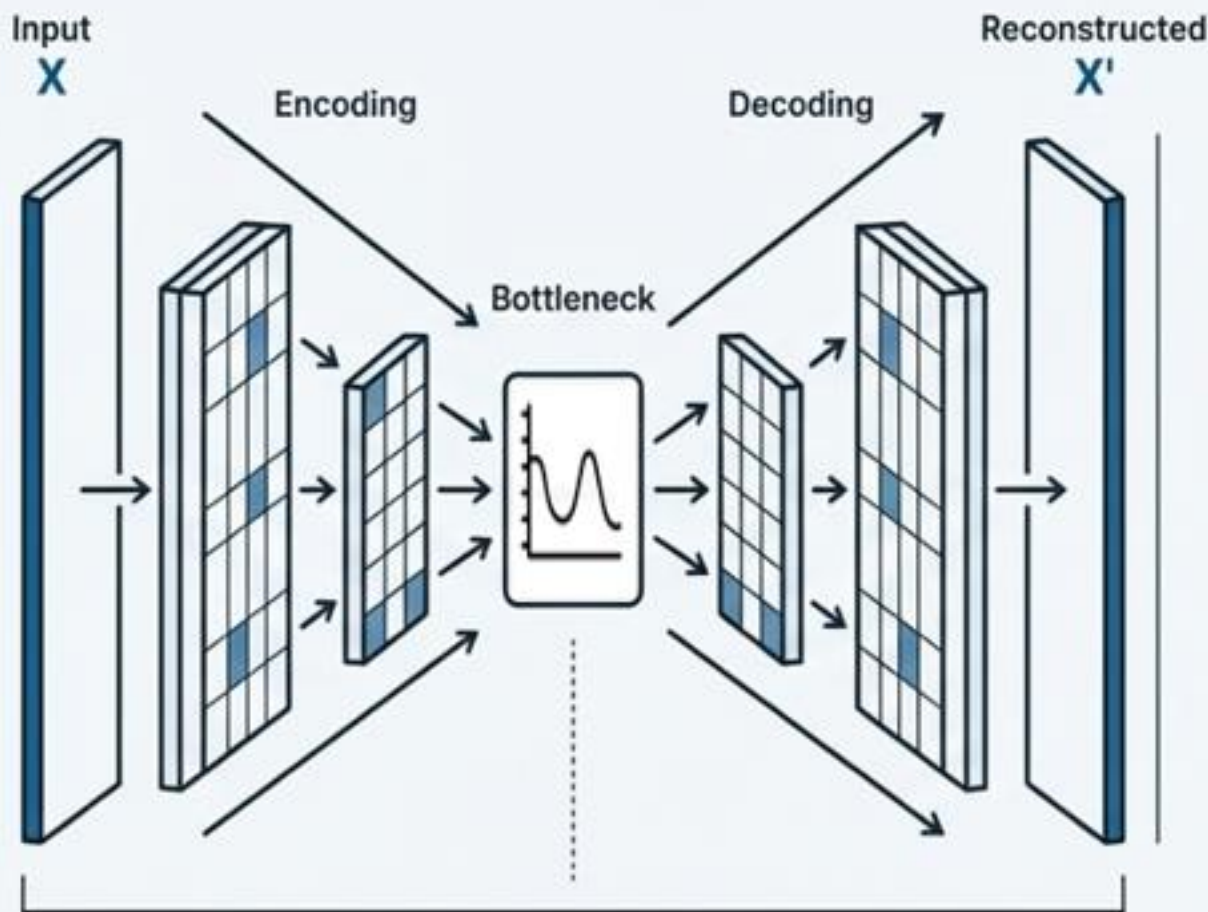


Step 2. 모델링: 정상 상태의 '패턴'을 학습하는 AutoEncoder

- AutoEncoder는 입력을 압축했다가 다시 원본으로 복원하도록 학습하는 신경망입니다.

학습 원리: '정상' 데이터만으로 학습하면, 모델은 정상 패턴을 복원하는 방법을 특화하여 학습합니다.

이상탐지 메커니즘: 학습된 모델에 '이상' 데이터가 입력되면, 모델은 이를 제대로 복원하지 못하고 큰 '재구성 오차(Reconstruction Error)'를 발생시킵니다. 이 오차값이 바로 이상 점수가 됩니다.

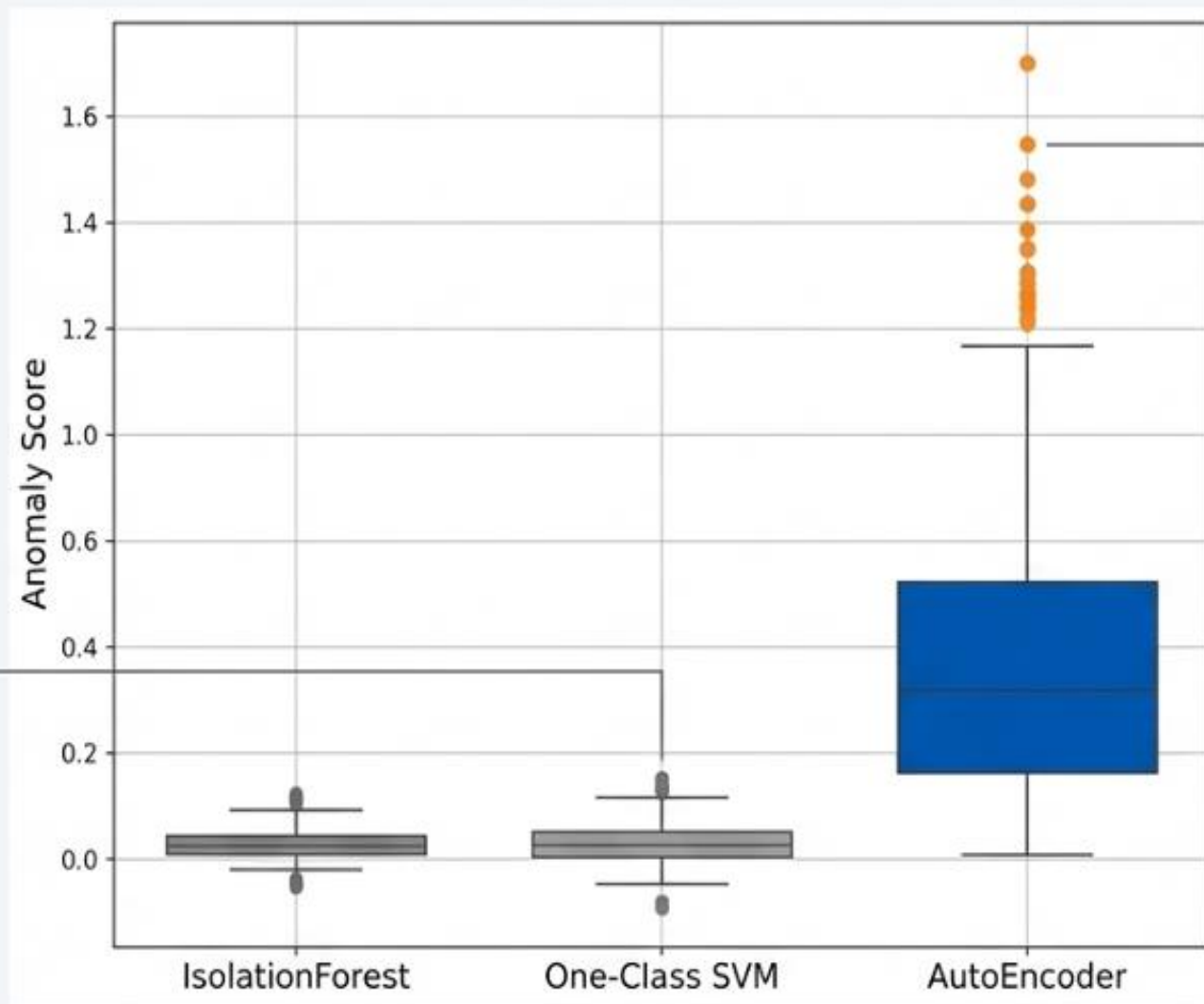


$$\text{Anomaly Score} = \| X - X' \|$$

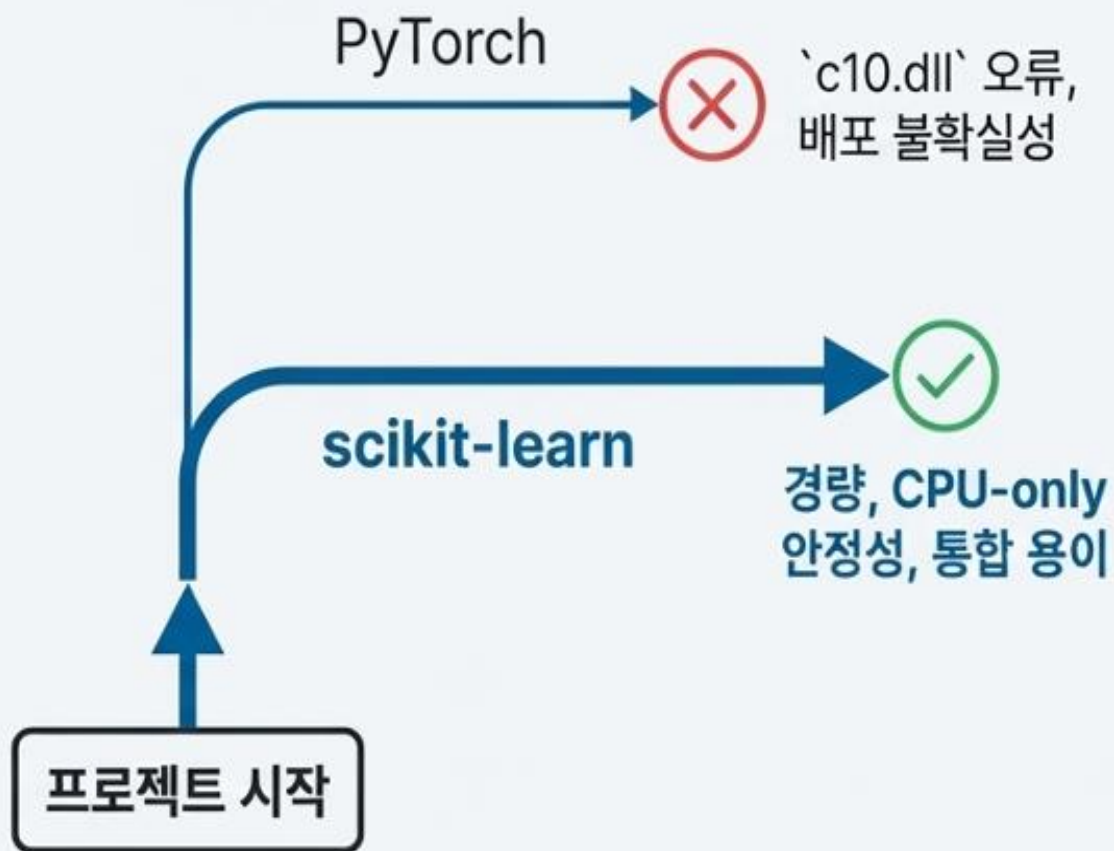
데이터가 증명합니다: 왜 AutoEncoder가 유일한 선택인가

세 가지 비지도학습 모델의 이상 점수(Anomaly Score) 분포 비교

분포가 좁고 대부분의 샘플을 유사하게 판단합니다.
정상과 이상의 분리도가 매우 낮아 미세한 패턴 차이를 감지하기 어렵습니다.



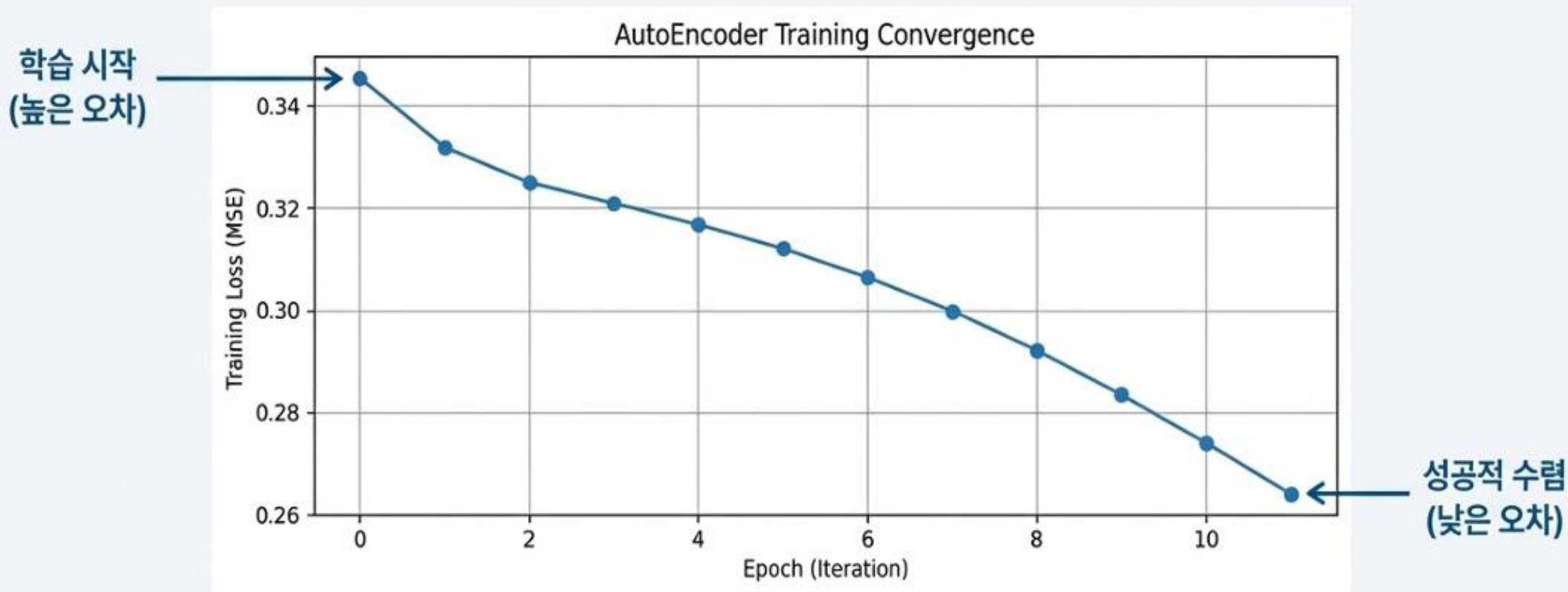
정상/이상 간 점수 스케일 차이가 명확합니다.
분산이 크고 상단에 이상치(outlier)가 뚜렷하게 나타나, 임계값 설정에 매우 유리합니다.



현실적 제약 극복: 저사양 환경에서의 안정성을 위한 기술 전환

- **초기 문제:** 프로젝트 초기, PyTorch 기반 모델 구현 시 저사양 PC 환경에서 특정 DLL (`c10.dll`) 로딩 오류가 발생했습니다. 이는 CPU 명령어셋 의존성 문제로, 배포 환경의 불확실성을 높이는 요인이었습니다.
- **전략적 결정:** 높은 사양의 하드웨어나 복잡한 의존성 없이도 안정적으로 동작하는 것이 중요하다고 판단, **scikit-learn의 MLPRegressor**를 이용한 AutoEncoder로 전환했습니다.
- **결과:** 이 결정 덕분에, 최종 AI 모듈은 **CPU-only** 환경에서도 가볍고 안정적으로 동작하여 실제 시스템 통합 시의 기술적 장벽을 크게 낮추었습니다.

학습 과정 검증: 모델은 성공적으로 수렴되었습니다



- 모델 학습은 단순히 코드를 실행하는 것을 넘어, 학습이 올바르게 진행되고 있는지를 확인하는 과정이 중요합니다.
- scikit-learn 모델의 `loss_curve`를 시각화하여, 학습 반복(Iteration)이 진행됨에 따라 재구성 오차(MSE Loss)가 꾸준히 감소하며 안정적으로 수렴했음을 확인했습니다.
- 과적합 방지를 위해 `early_stopping`을 적용하여, 일정 횟수 이상 성능 개선이 없을 경우 자동으로 학습을 중단하도록 설계했습니다.

객관적 판단 기준: 정상 데이터 분포 기반의 통계적 임계값(p99) 설정

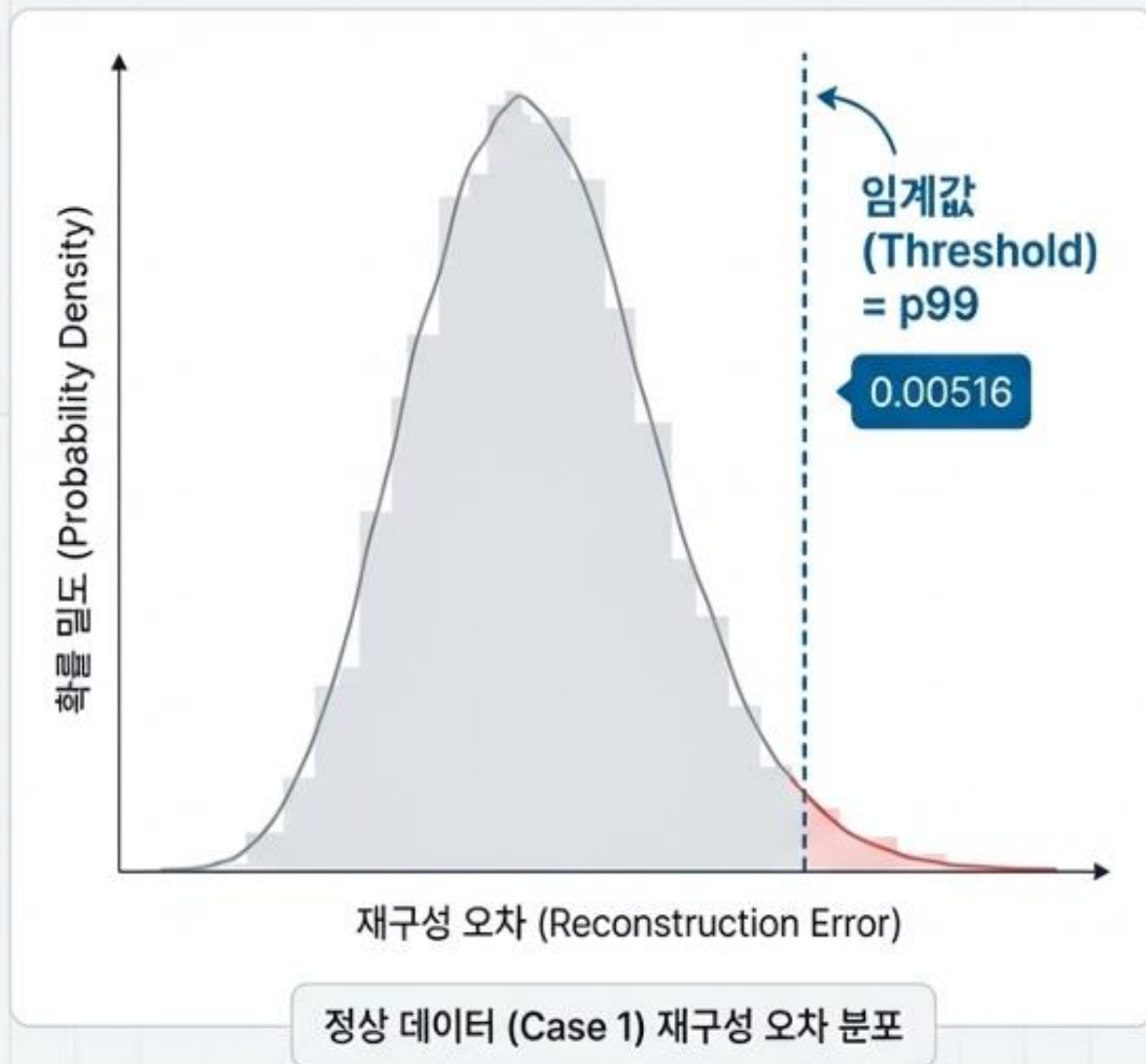
- 이상 여부를 판단하는 임계값(Threshold)은 임의로 설정하는 것이 아니라, 데이터에 기반한 통계적 기준으로 정의해야 합니다.

정의 방식:

1. '정상' 상태(Case1)의 학습 데이터 전체에 대해 재구성 오차를 계산합니다.
2. 계산된 오차 값들의 분포에서 **상위 1%에 해당하는 99퍼센타일(p99) 값**을 임계값으로 설정합니다.

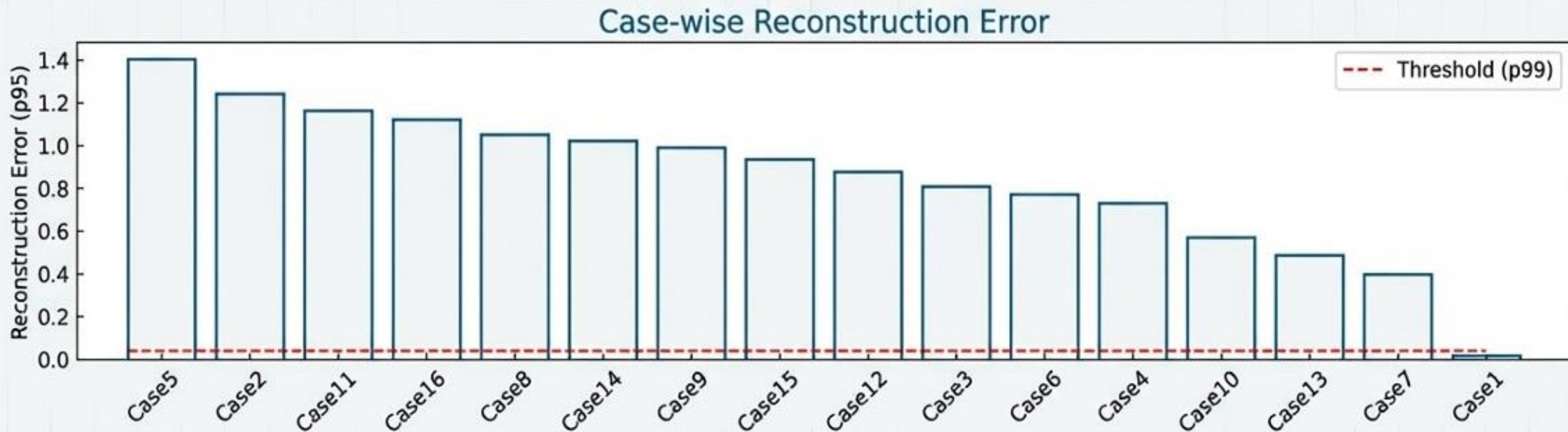
의미: 이 방식은 "정상 데이터에서 발생할 수 있는 오차의 99%는 이 값보다 작다"는 통계적 근거를 가지므로, 매우 설득력 있고 객관적인 판단 기준이 됩니다.

설정된 임계값: 0.00516



최종 결과: 모델이 정상과 이상 상태를 명확하게 구분합니다

- 전체 16개 케이스에 대해 이상 점수(Reconstruction Error p95)를 계산하고, 정상 데이터 기반으로 설정된 임계값(Threshold p99)과 비교했습니다.
- 결과적으로, 정상 상태인 `Case1`의 오류는 임계값보다 현저히 낮은 반면, 다수의 이상 케이스들은 임계값을 수십-수백 배 초과하는 높은 오류를 기록했습니다.
- 이는 우리 모델이 볼트 풀림 상태를 매우 효과적으로 탐지하고 있음을 시각적으로 증명합니다.



최종 모델 테스트

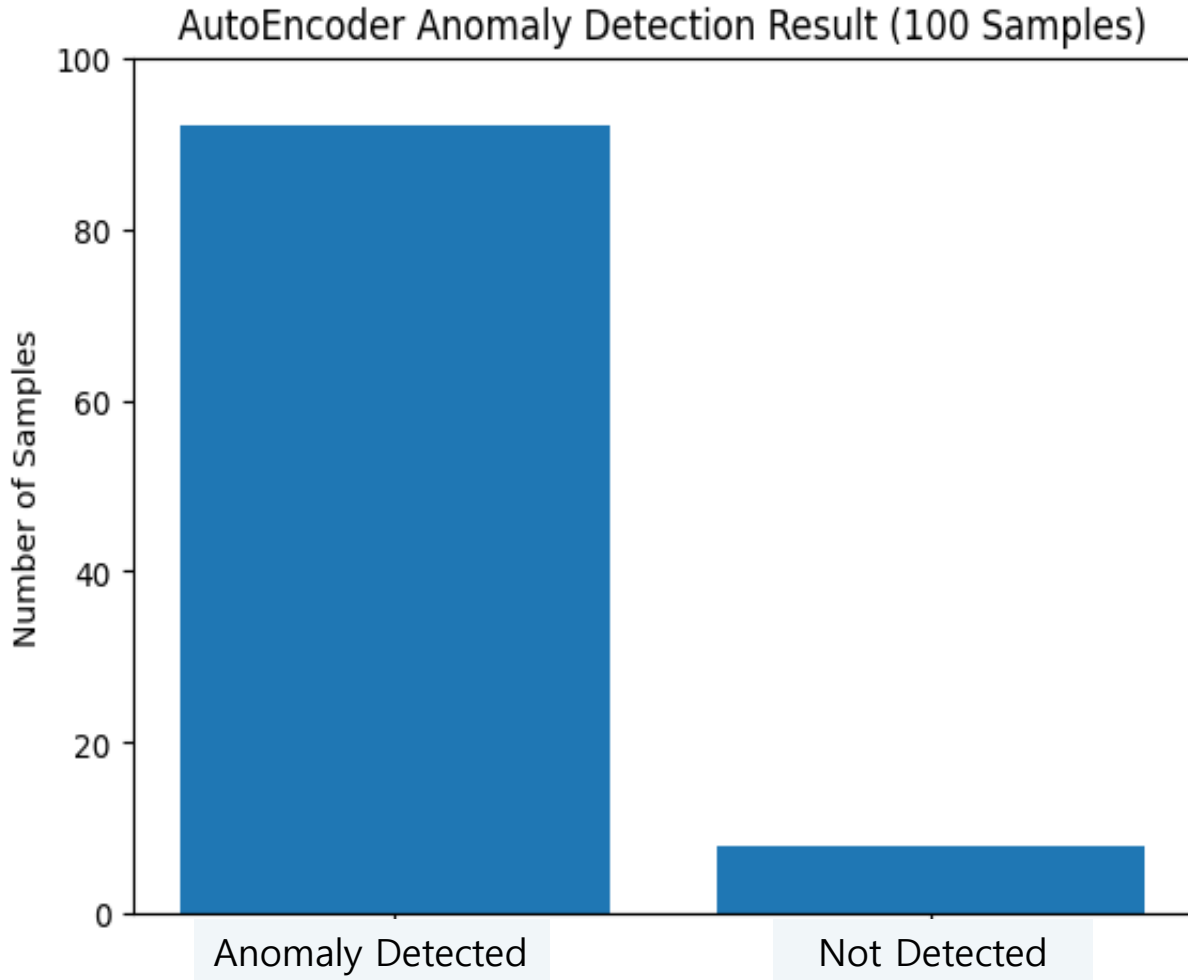
- 본 기술의 신뢰도 평가를 위해서는 실제 이상 상황 데이터를 활용한 검증이 필요합니다.
- 그러나 항공기, 로켓, 정밀 로봇 등 고신뢰·고위험 산업 도메인에서는 의도적인 결함 발생 실험이 현실적으로 어렵습니다.
- 이에 따라 본 연구에서는 실제 정상 운용 데이터를 기반으로 이상 특성이 반영된 가상 데이터를 생성하여 테스트를 수행하였습니다.
- 총 100개의 이상(볼트 풀림) 데이터에 대해 실험을 수행한 결과, **92개의 이상을 정확히 탐지하였습니다.**
- 이를 통해 제안한 모델의 이상 탐지 성능과 신뢰성을 확인하였습니다.

```
===== RESULT =====  
Reconstruction error: 0.0100007495  
Threshold: 0.01761365495622158  
Anomaly: 0
```

그림1. 미탐지
Threshold=0.017

```
===== RESULT =====  
Reconstruction error: 0.1716707  
Threshold: 0.01761365495622158  
Anomaly: 1
```

그림2. 탐지
Threshold=0.017



Sample 100개 중 92개 탐지

결과 해석: 높은 재구성 오차는 '학습한 적 없는 낯선 패턴'의 발견을 의미합니다

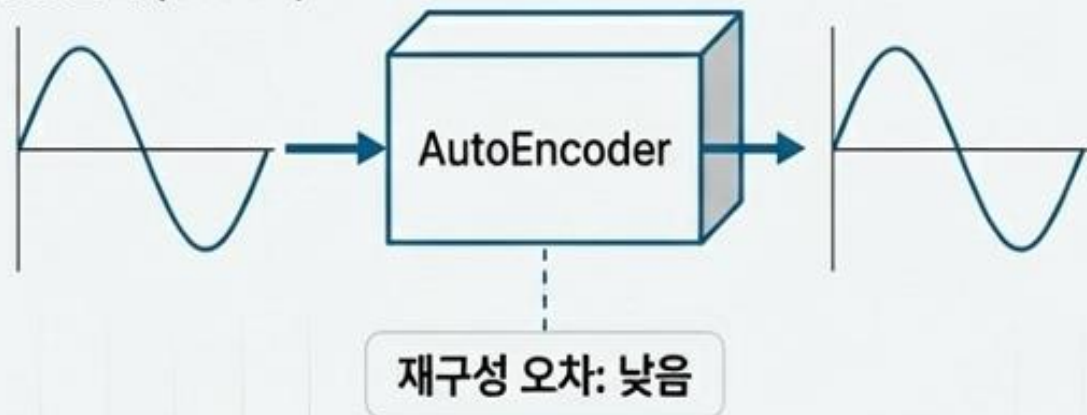
- 왜 이상 케이스의 오차가 높은가?

- 모델은 '정상(Case1)'의 진동 패턴만을 완벽하게 재구성하도록 학습되었습니다.
- 볼트가 풀린 케이스의 진동 신호는 정상 패턴과 근본적으로 다르기 때문에, 모델이 이를 제대로 재구성하지 못하고 큰 오차를 발생시키는 것입니다.

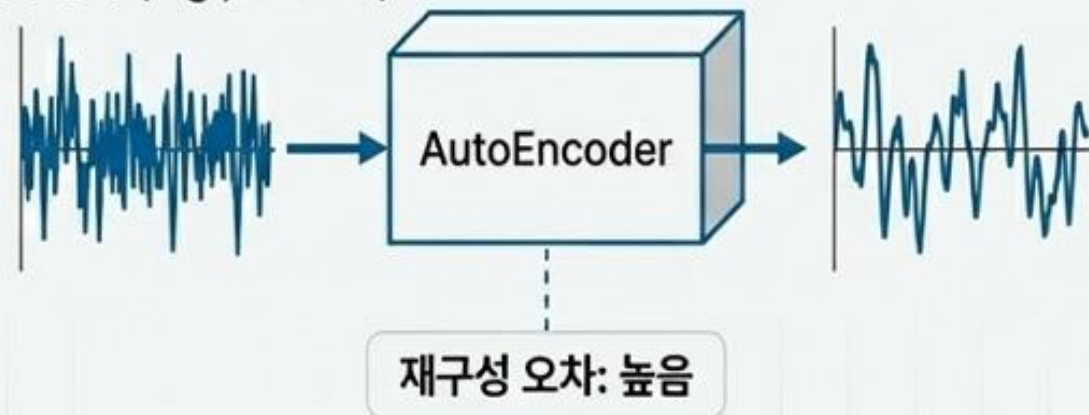
- 핵심: 이 "재구성의 실패"가 바로 "정상에서 벗어난 구조적 이상을 강하게 포착했다"는 성공적인 신호입니다.

- 성능: 임계값(~ 0.005) 대비 이상 케이스의 오차($1.0 \sim 1.4$)가 매우 커서, 정상과 이상의 분포가 거의 겹치지 않는 강력한 이상탐지 성능을 보여줍니다.

정상 입력 (Case 1)



이상 입력 (e.g., Case 5)



이론을 넘어 실제 적용으로: 즉시 통합 가능한 AI 모듈 패키지

- 본 프로젝트의 최종 목표는 논문이 아닌, 개발자가 즉시 시스템에 통합할 수 있는 실용적인 AI 모듈을 제공하는 것입니다.
- 이를 위해, 추론에 필요한 모든 자산을 다음과 같이 패키지화하여 전달합니다.



ai_assets/ae_sklearn.npz

학습된 모델 가중치



ai_assets/threshold.json

통계적 판단 임계값



ai_assets/feature_stats.npz

정규화에 필수적인 평균/표준편차



ai_assets/inference.py

입력 데이터로부터 이상 점수와
판정 결과를 반환하는 핵심 함수



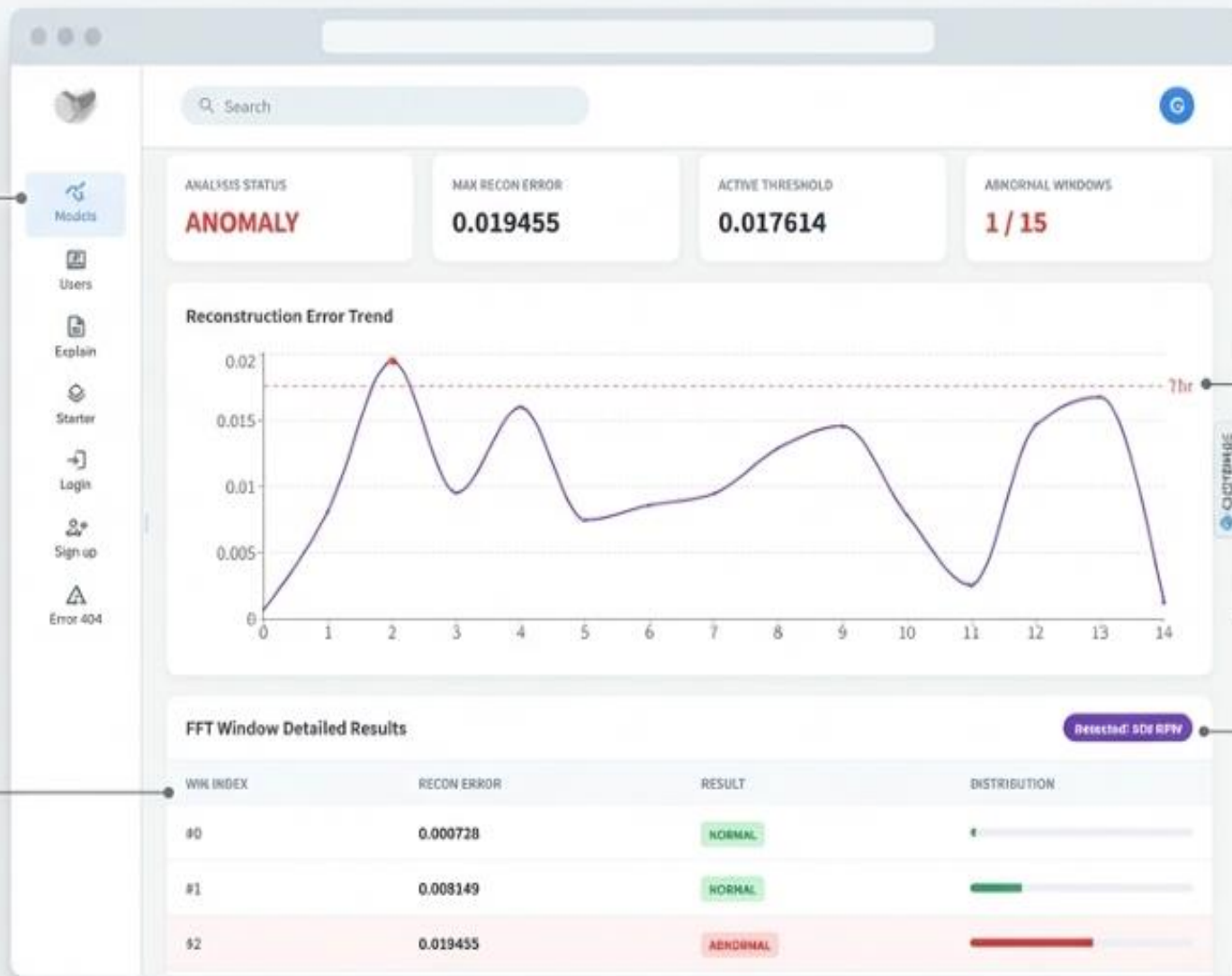
ai_assets/README.md

API 명세 및 사용법을 기술한
개발자용 가이드

그리하여, 'NotNut'이 탄생했습니다

입증된 모델을 기반으로 구현된 실시간 이상 탐지 대시보드

즉각적인 상태 분석:
ANOMALY/NORMAL
상태 즉시 확인

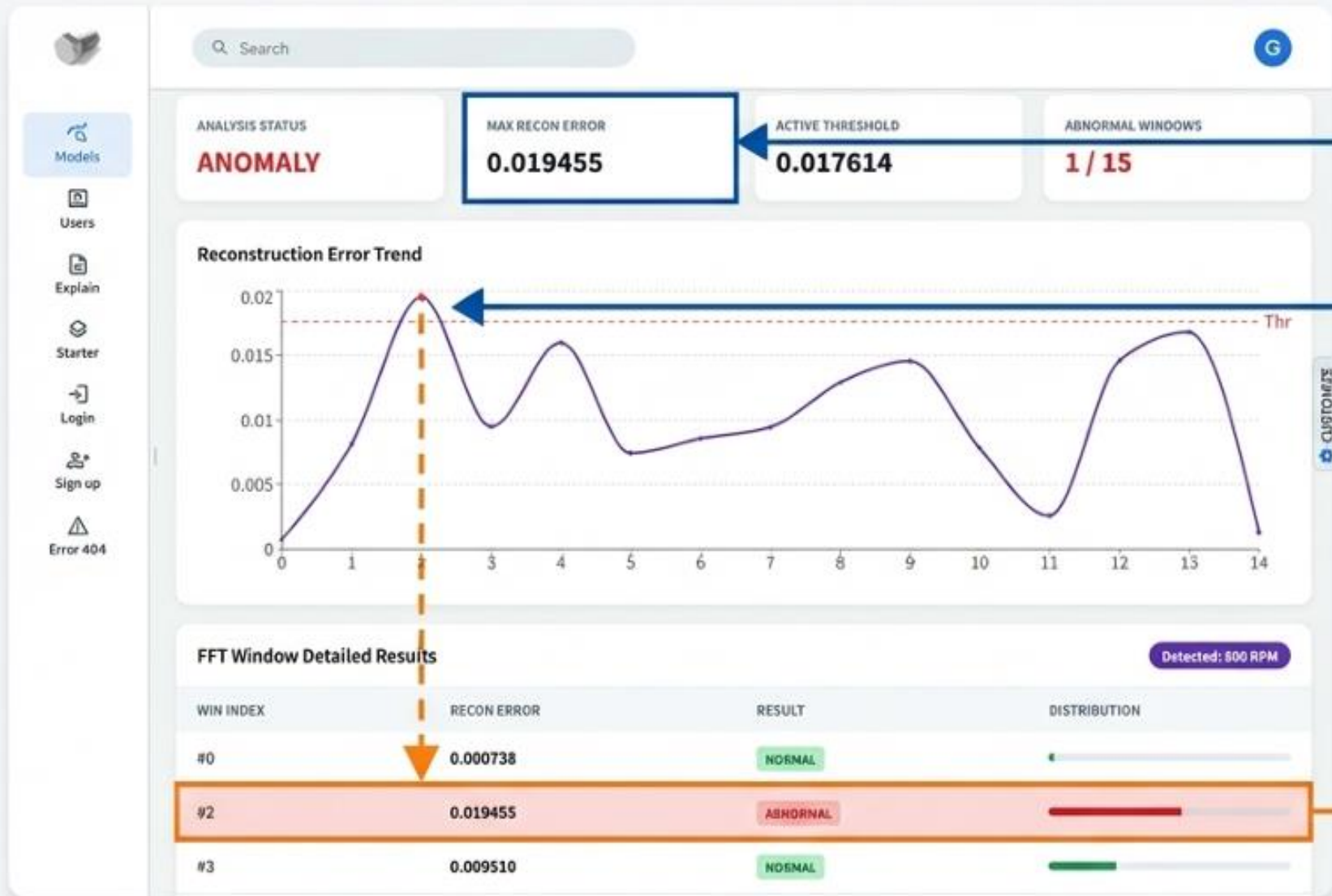


상태 분석 결과:
개별 데이터 윈도우 별
오차 수치 및 정보 제공

오차범위 시각화:
입력된 데이터 중 오차가
발생한 부분 시각화

조건별 모델 적용:
현재 분석 중인 RPM(e.g.,
Detected: 800 RPM) 표시

하나의 이상 징후를 포착하는 과정



2. 최대 오차 확인

전체 분석 구간에서 가장 높은 오차 값 (0.019455)을 파악합니다

1. 트렌드 분석

복원 오차(Reconstruction Error)가 설정된 임계값(Threshold)을 초과하는 지점을 시각적으로 확인합니다.

3. 이상 구간 특정

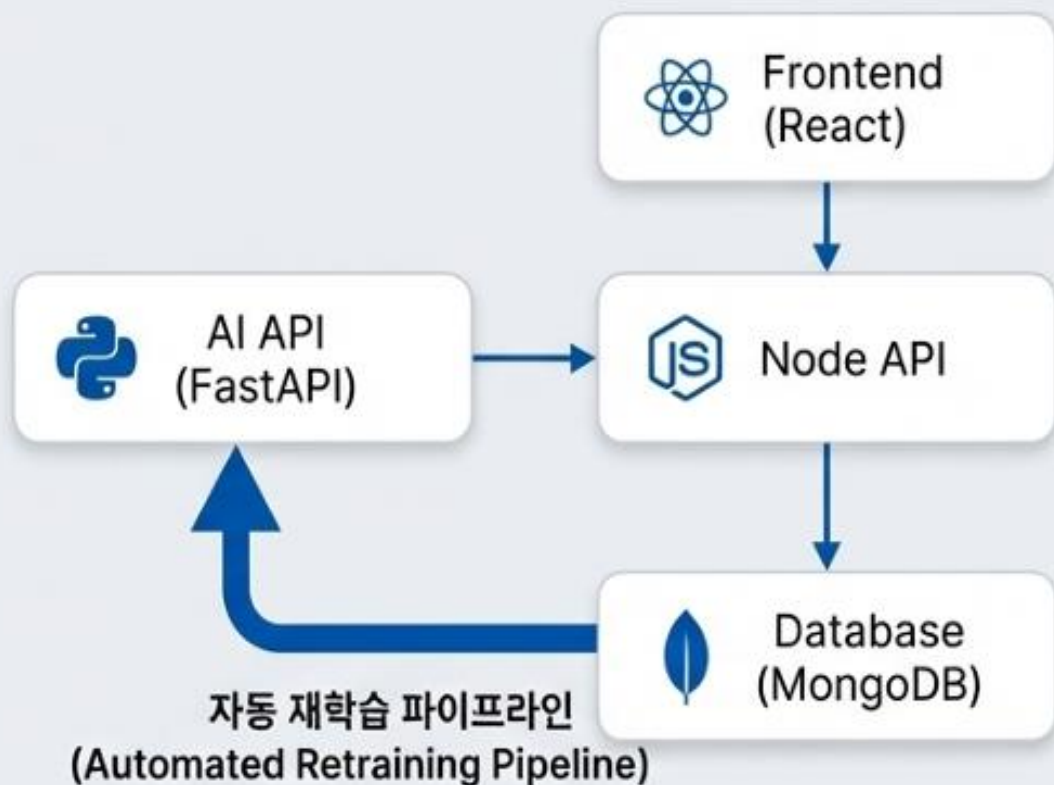
오차가 급증한 윈도우 인덱스를 특정하고, 해당 구간이 'ABNORMAL'로 판별되었음을 확인합니다.

시스템은 단순히 임계값을 넘었는지 뿐만 아니라, 어느 시점에서 어떤 값으로 이상이 발생했는지에 대한 상세한 분석 근거를 제공합니다.

단순한 모델을 넘어, 지속적으로 진화하는 시스템으로

MLOps 파이프라인이 적용된 시스템 아키텍처

Docker Compose Orchestration



컨테이너 기반 아키텍처

Docker & Docker Compose를 활용하여 전체 인프라의 안정성과 배포 용이성을 확보했습니다.

데이터 품질 관리

- **유사성 검증:** 신규 데이터가 기존 데이터 패턴과 현저히 다를 경우, DB에 저장하지 않고 추론만 수행하여 데이터 무결성을 유지합니다.
- **중복 방지:** SHA-256 해시를 사용하여 동일 파일의 중복 업로드를 차단합니다.

MLOps 자동화: 자동 재학습 파이프라인

특정 RPM의 유효 데이터가 100개 수집될 때마다 **자동으로 모델 재학습을 트리거**하여 최신 데이터 경향을 반영하고 성능을 유지합니다.

MLOps 파이프라인 세부 설명



1. 데이터 입력

- 사용자는 진동 데이터가 포함된 CSV 파일을 웹 프론트엔드를 통해 업로드합니다.
- 업로드 단계에서는 파일 형식 및 기본 유효성 검사를 수행합니다.

2. 입력 데이터 강제 파싱 및 유사성 검사

- AI API 서버에서 CSV의 첫 번째 컬럼을 강제로 추출하여 분석합니다.
- 학습 데이터와의 유사성 검사를 통해 정상 입력 여부를 사전 판단합니다.
- 유사성 검사를 통과하지 못한 데이터는 추론만 수행하며, 저장하지 않습니다.

3. 추론 및 결과 저장

- 유사성 검사를 통과한 데이터에 대해서만 이상 탐지 추론을 수행합니다.
- 추론 결과 및 핵심 지표는 MongoDB에 저장됩니다.

4. 실시간 모니터링

- 누적 데이터가 증가함에 따라 다음 지표를 지속적으로 모니터링합니다.
 - Reconstruction Error 분포
 - Threshold Crossing Rate 변화
 - 오차 분포 형태 변화

5. 재학습 및 모델 교체 판단

- 데이터가 100건 이상 누적되거나, 이상 분포의 통계적 형태 변화가 감지될 경우 재학습을 수행합니다.
- 재학습 결과 기존 모델 대비 성능이 개선될 경우 모델을 교체합니다.

The image shows the 'NotNut' logo in the top left corner, featuring a small nut icon and the text 'NotNut' in orange and green. Below the logo is a large illustration of three anthropomorphic nut characters (two green, one blue) carrying a very large silver bolt horizontally across their backs. The bolt has a hexagonal head on the left and a threaded section in the middle. The nut characters are smiling and walking towards the right.

3

 테마 설정 ☐

 언어 설정: 한국어

 계정 및 보안

 로그아웃

설정 페이지

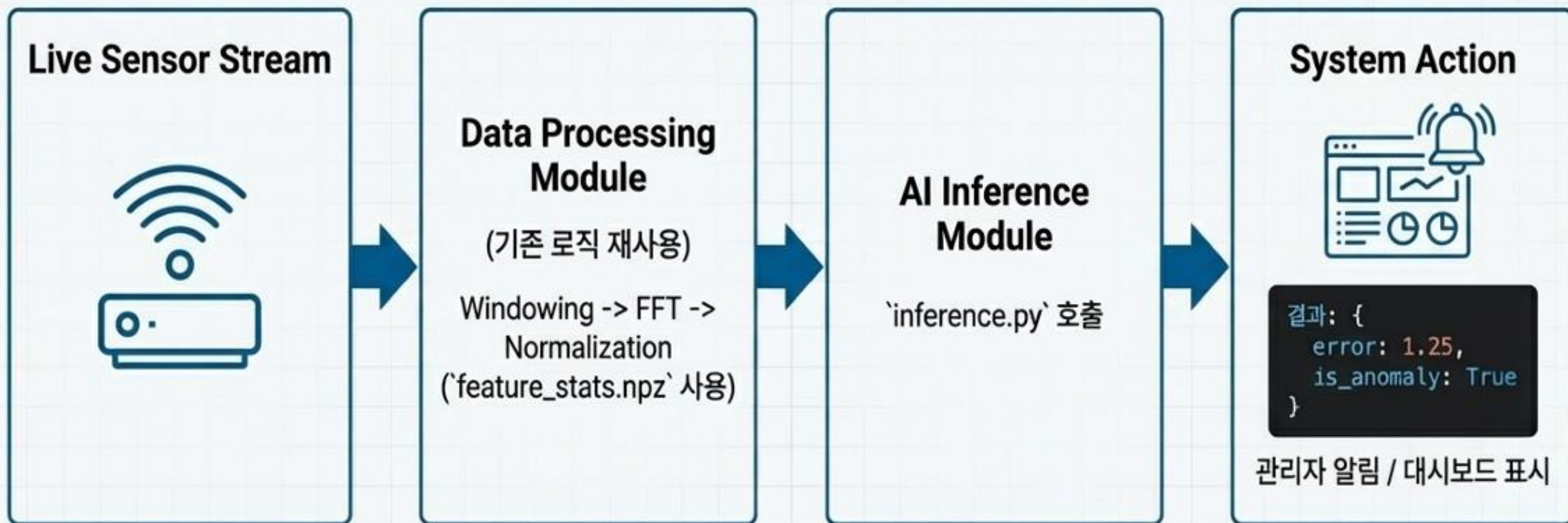
 계정 및 보안

프로필 관리	
닉네임	현재 닉네임 3 새 닉네임 입력
계정 정보 변경 (ID/이메일)	변경할 새 User ID 새 이메일 주소
비밀번호 변경	기존 비밀번호 입력 새 비밀번호 입력
Input Model 리스트 (CSV 관리)	파일명: Case6_1000.csv 결과내용: ANOMALY 결과

설정페이지

실시간 시스템으로의 확장 경로

- 완성된 AI 모듈은 입력 데이터 단위로 독립적인 판단이 가능한 `stateless` 구조로 설계되어, 다양한 시스템에 쉽게 통합할 수 있습니다.
- 실제 센서가 도입되면, 아래와 같은 흐름으로 실시간 이상 감지 시스템을 즉시 구축할 수 있습니다.



PoC 성공: AI 판단 로직 검증 완료, 시스템 통합 리스크 최소화

본 과제를 통해 달성한 핵심 성과:

문제 해결 (Problem Solved)



정상 데이터만으로 볼트 풀림을 탐지하는 AutoEncoder 이상탐지 모델 구축에 성공했습니다.

성능 검증 (Model Validated)



통계적 기준(p99)과 명확한 케이스별 오차 분리를 통해 모델의 높은 성능을 객관적으로 입증했습니다.

통합 준비 (Ready for Integration)



실제 시스템에 즉시 적용 가능한 경량 AI 모듈과 개발자용 산출물 패키지를 완성했습니다.

의의: 본 PoC는 실제 센서 도입 및 시스템 개발 이전에 **핵심 AI 판단 로직의 불확실성을 완전히 제거**하여, 향후 프로젝트의 리스크를 크게 낮추었습니다.

NotNut: 누구를 위한 기술인가

“이 기술은 모든 현장을 위한 기술이 아니라,
한 번의 실패가 수백억의 손실로 이어지는
최첨단 산업을 위한 기술입니다.”

**볼트 풀림 이상탐지 AI AI 모듈,
지금 바로 실시간 시스템에 적용 가능합니다.**



Appendix A: 시스템 설계 및 데이터 전처리 원칙

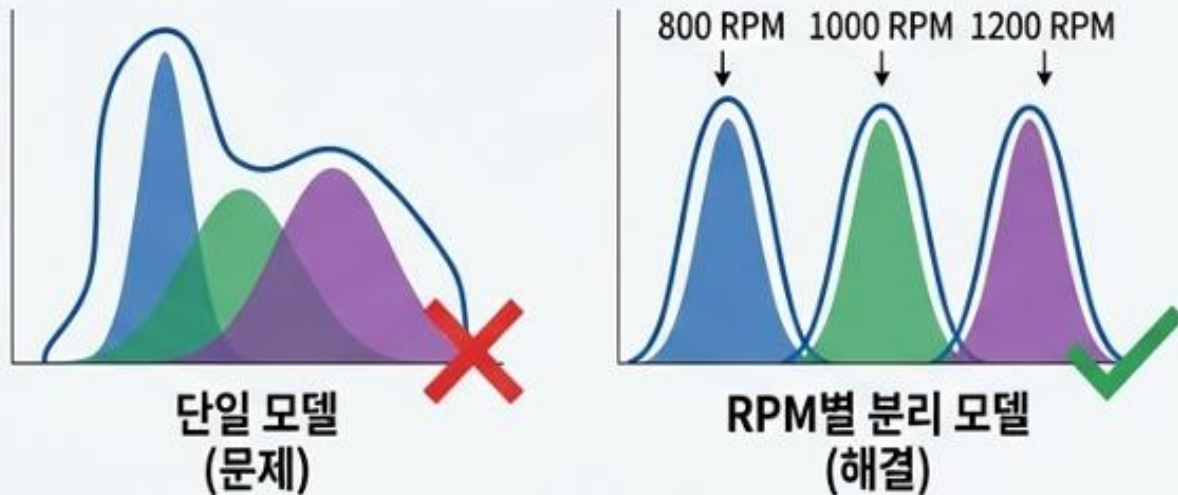
RPM별 모델 분리 설계 이유

문제점

RPM(회전 속도) 변화는 시스템의 동적 특성 자체를 변화시켜, 동일한 결함이라도 주파수 분포가 완전히 달라집니다.

해결책

800/1000/1200 RPM 등 운전 조건별로 모델을 분리하여, 단일 모델의 평균화된 판단 위험을 제거하고 임계값 안정성과 탐지 정확도를 극대화했습니다.



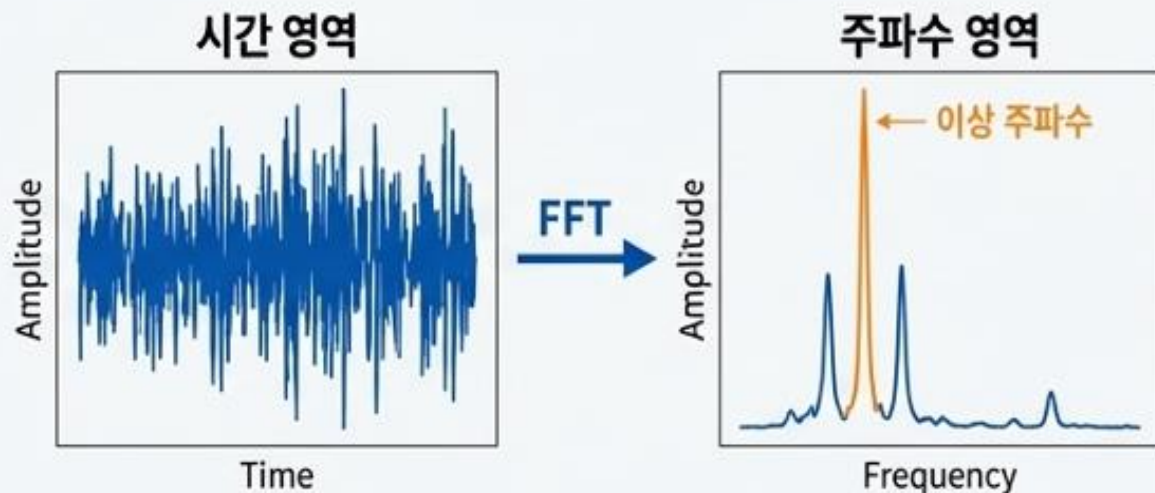
주파수 영역(FFT) 특징 추출의 중요성

FFT의 필요성

기계 진동의 이상 징후(볼트 풀림 등)는 시간 영역 신호보다 특정 주파수 대역의 에너지 변화로 더 명확하게 나타납니다.

이점

FFT 기반 스펙트럼 변환은 딥러닝 모델이 물리적으로 의미 있는 패턴을 학습하도록 돕는 핵심 전처리 과정입니다.



Appendix B: 왜 AutoEncoder 모델링이 최적의 접근법인가

왜 분류(Classification) 모델이 아닌가?

실제 산업 현장에서는 모든 고장 유형을 사전에 정의하고 데이터를 수집하는 것이 불가능하며, 정상 데이터 대비 고장 데이터가 압도적으로 부족합니다.

다른 비지도 이상탐지 기법과의 비교

기법 (Method)	주요 한계 (Key Limitation)	NotNut 적용 시 문제점 (Issue for NotNut)
Rule-based	환경 변화에 취약	미세한 변화 및 새로운 유형의 이상 탐지 불가
통계 기반 (Statistical)	비선형/복잡 패턴 인식 한계	복잡한 진동 신호의 구조적 변화를 놓침
Isolation Forest	고차원/연속적 데이터 구조에 취약	주파수 스펙트럼과 같은 고차원 파형 데이터의 분리도 낮음
AutoEncoder	(선택 이유) 비선형/고차원/무감독(Unsupervised) 특징을 모두 만족하며, 사람이 정의하지 못하는 이상을 스스로 학습 가능	

Appendix C: 한계점 및 향후 개선 방향

현재 모델의 한계 (Current Model Limitations)



Limited Fault Data: 현재 검증은 제한된 유형의 이상(N=1) 데이터를 기반으로 수행되었습니다.



Environmental Noise: 온도, 습도 등 외부 환경 잡음 변수는 현재 모델에서 고려되지 않았습니다.

향후 연구 및 개발 방향 (Future Research & Development Roadmap)



Multi-sensor Fusion: 진동, 음향, 온도 등 여러 센서 데이터를 융합하여 탐지 정확도를 높이는 모델을 연구할 것입니다.



Advanced Models: 시계열 패턴 인식에 더 강력한 Transformer 기반 AutoEncoder 모델 적용을 검토합니다.



Dynamic RPM Handling: 특정 RPM 값이 아닌, 연속적인 RPM 변화에 대응할 수 있는 모델을 개발하여 적용 범위를 확장할 것입니다.

감사합니다